SQL Server 2022
Intelligent Query Processing

DATA SATURDAYS

Innova community

Sponsors

With the support of:

About me

- Gianluca Hotz | @glhotz | ghotz@ugiss.org

- Independent Consultant
  - 25+ years on SQL Server (from 4.21 back in 1996)
  - Database modeling & development, sizing & administration, modernization (upgrades & migrations), performance tuning, security

- Community
  - 24 years Microsoft MVP SQL Server/Data Platform (from 1998)
  - VMware Experts SQL Server
  - Founder and president UGISS (ex «PASS Chapter»)

DATA SATURDAYS

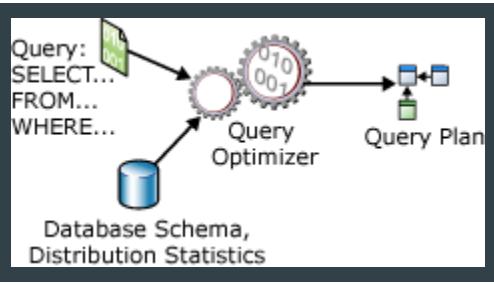
Introduzione

SQL Server 2022 Intelligent Query Processing

With the support of:

innOva community · altitudo · beanTech IT moves your business · fluentis the ERP ready to go live! · QUANTUMDATIS · [stesi] Powered by Innovation · CREDEM · CREDEMTEL · packt

«Query Optimizer»

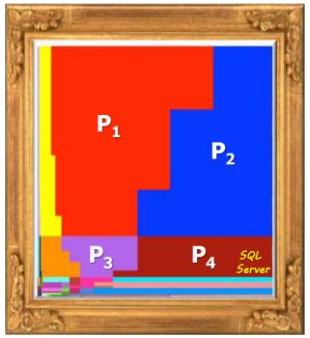


- SQL è un linguaggio dichiarativo
  - Specifichiamo cosa vogliamo ottenere, non come
- Il QO genera un piano di esecuzione
  - Il come ottenere le cose richieste
  - Tipicamente cercandolo tra molteplici piani possibili
  - Sfruttando modello basato su costi e statistiche distribuzione valori
  - https://docs.microsoft.com/en-gb/sql/relational-databases/query-processing-architecture-guide

With the support of:

Un po' di storia

- Ottimizzazione «cost-based» risale 1970
  - Pat Selinger in System R (poi divenuto DB2)
- 50 anni, RDBMS ancora software più difficile da implementare
  - Paura di causare regressioni
- Obiettivo QO non miglior piano in assoluto…
  - Spazio delle soluzioni troppo grande per essere esplorato
  - Miglior piano in un tempo ragionevole (approccio euristico)
  - Sceglie ancora troppi piani inefficienti ☹
- Problemi comuni
  - Statistiche non accurate (piani inefficienti)
  - Distribuzione anomala («Parameter Sniffing» e nessun piano ottimale)
  - Assunzioni correlazione flitri/JOIN (nuovo «Cardinality Estimator»)

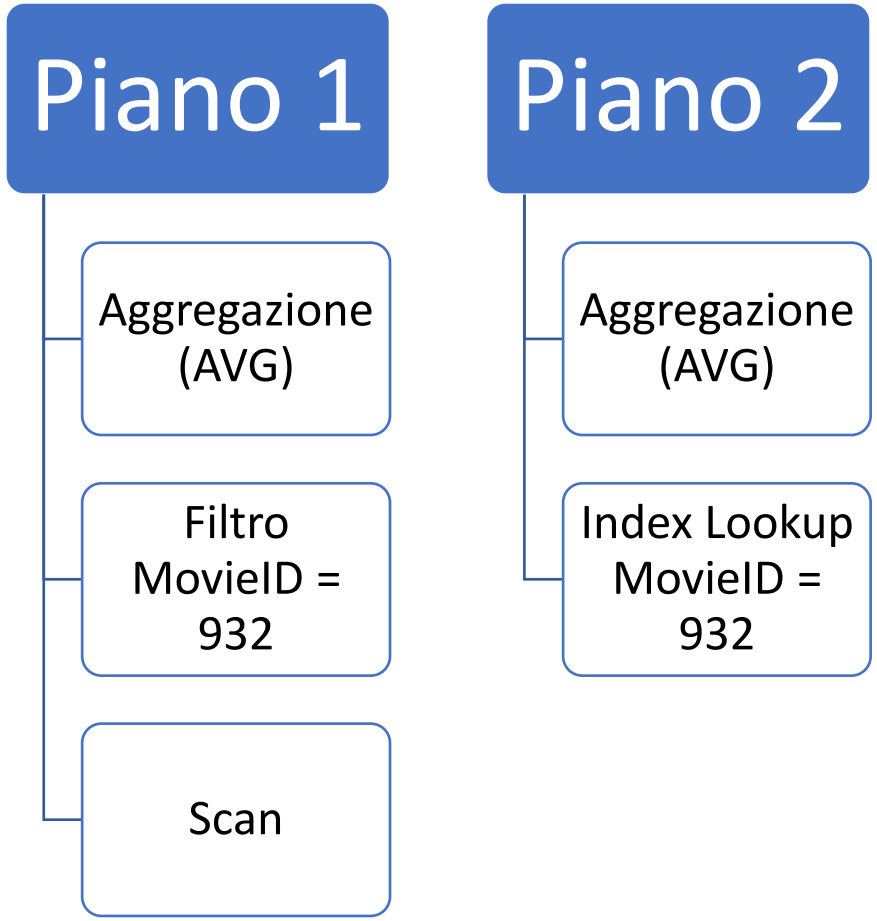


With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
<packt>

Esempio

```
SELECT AVG(Rating)
 FROM   Reviews
WHERE  MovieID = 932
```

- Calcolare la **media** del **punteggio** delle **recensioni** del **film** con **identificativo 932**



With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
packt

Esempio più complesso

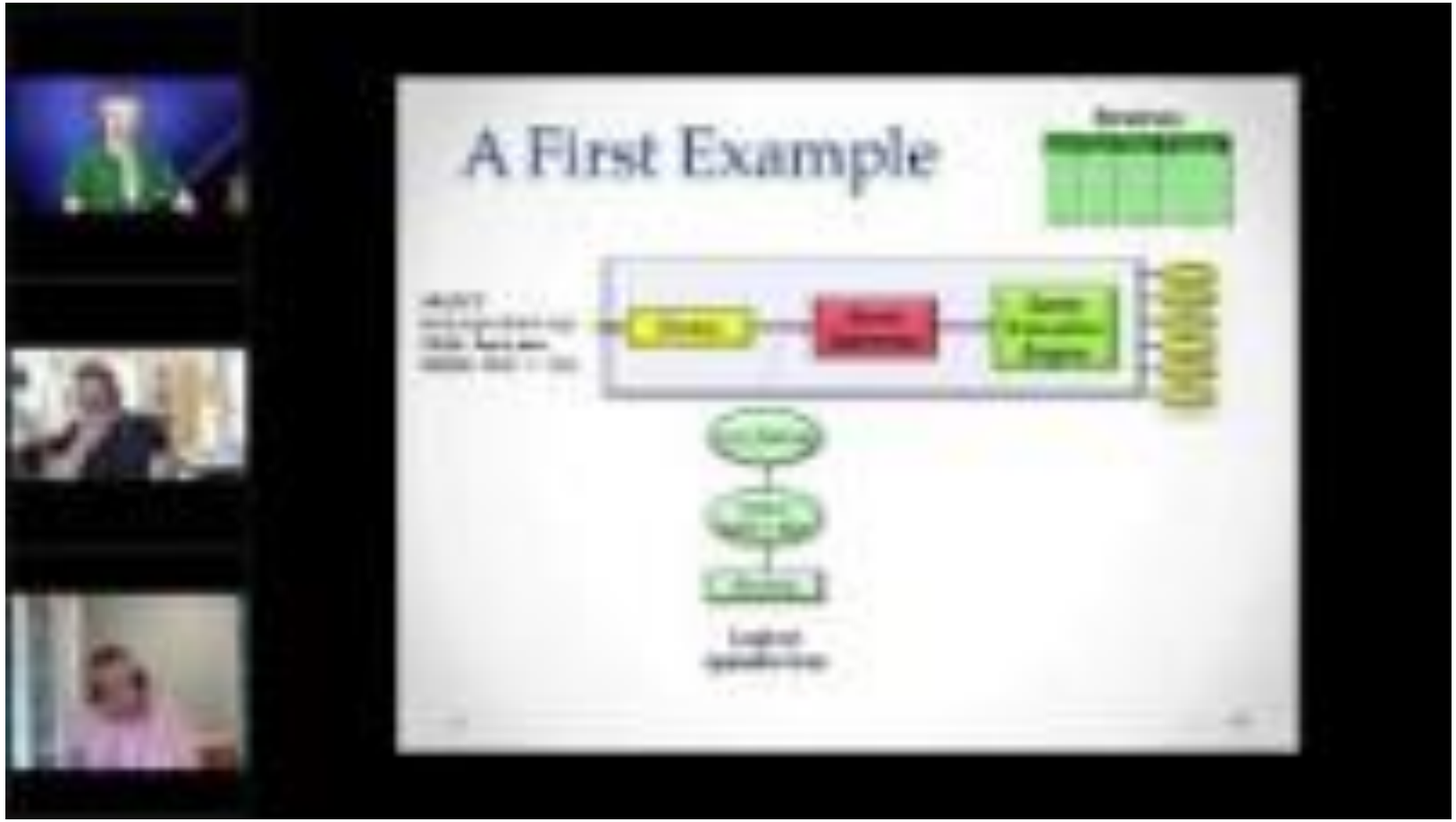
- TPC H Query 8
  - National Market Share
- Circa 22 milioni di piani!

```
Select o_year,
sum(case
 when nation = 'BRAZIL' then volume
 else 0
end) / sum(volume)
from
(
 select YEAR(O_ORDERDATE) as o_year,
  L_EXTENDEDPRICE * (1 - L_DISCOUNT) as volume,
  n2.N_NAME as nation
 from PART, SUPPLIER, LINEITEM, ORDERS, CUSTOMER, NATION
n1,
 NATION n2, REGION
 where
    P_PARTKEY = L_PARTKEY and S_SUPPKEY = L_SUPPKEY
    and L_ORDERKEY = O_ORDERKEY and O_CUSTKEY = C_CUSTKEY
    and C_NATIONKEY = n1.N_NATIONKEY and n1.N_REGIONKEY =
R_REGIONKEY
    and R_NAME = 'AMERICA' and S_NATIONKEY = n2.N_NATIONKEY
    and O_ORDERDATE between '1995-01-01' and '1996-12-31'
    and P_TYPE = 'ECONOMY ANODIZED STEEL'
    and S_ACCTBAL <= constant-1
    and L_EXTENDEDPRICE <= constant-2
) as all_nations
group by o_year  order by o_year
```

With the support of:

DATA SATURDAYS

SQL Query Optimization: Why Is It So Hard to Get Right?



With the support of:

innOva community altitudo beanTech IT moves your business fluentis the ERP ready to go live! QUANTUMDATIS [stesi] Powered by Innovation CREDEM CREDEMTEL <packt>

Strumenti per affrontare il problema

- Query Hints
- Query Store
- Intelligent Query Processing

DATA SATURDAYS

Query Store

SQL Server 2022 Intelligent Query Processing

With the support of:

innOva community · altitudo · beanTech IT moves your business · fluentis the ERP ready to go live! · QUANTUMDATIS · [stesi] Powered by Innovation · CREDEM · CREDEMTEL · packt

Query Store 101

- Registra piani di esecuzione e metriche prestazioni
  - Per database
  - In memoria consolidando su disco in modalità asincrona
  - Integrato con SSMS e disponibile via DMV
- Riduce drasticamente complessità «troubleshooting»
  - Identificare «Top Query» per tempo/memoria/CPU/IO
  - Analizzare utilizzo risorse per un determinato database
  - Audit storia piani di esecuzione di una determinata query
  - Identificare regressione piani di esecuzione
  - **Forzatura piano specifico da storico**
- Novità SQL Server 2017+
  - «Wait Stats» integrate (per categoria, no dettaglio)
- Novità SQL Server 2019+
  - Raccolta personalizzata (es. numero minino esecuzioni, durata minima ecc.)

SQL Server 2022 Query Store

- Attivo di default
  - database migrati mantengono stato originale)
- Supporto per repliche secondarie Availability Groups
- «Query Store Hints»
  - Forzatura piani esecuzione senza modifiche al codice
- Usato automaticamente da «Intelligent Query Processing»

«Query Store Hints»

- Processo più semplice di «Plan Guides»
  - Trovare «query id» nel Query Store
  - Usare **`sys.sp_query_store_set_hints`** per applicare «Hint»
  - Verificare problemi in **`sys.query_store_query_hints`**
- «Hint» sopravvive a riavvii e pulizia cache piani esecuzione
- Maggiori informazioni
  - https://learn.microsoft.com/sql/relational-databases/performance/query-store-hints

Supporto «Query Store Hints»

- **«Hint» supportati**
  - { HASH | ORDER } GROUP
  - { CONCAT | HASH | MERGE } UNION
  - { LOOP | MERGE | HASH } JOIN
  - EXPAND VIEWS
  - FAST number_rows
  - FORCE ORDER
  - IGNORE_NONCLUSTERED_COLUMNSTORE_INDEX
  - KEEP PLAN
  - KEEPFIXED PLAN
  - MAX_GRANT_PERCENT = percent
  - MIN_GRANT_PERCENT = percent
  - **MAXDOP** number_of_processors
  - NO_PERFORMANCE_SPOOL
  - OPTIMIZE FOR UNKNOWN
  - PARAMETERIZATION { SIMPLE | FORCED }
  - **RECOMPILE**
  - ROBUST PLAN
  - **USE HINT** ('<hint_name>' [, ...n])
- **«Hint» non supportati**
  - OPTIMIZE FOR(@var = val)
  - MAXRECURSION
  - USE PLAN
    - consider QP original plan forcing
  - DISABLE_DEFERRED_COMPILATION_TV
  - DISABLE_TSQL_SCALAR_UDF_INLINING
  - Table hints
    - E.g. FORCESEEK, READUNCOMMITTED, INDEX
- **Interoperabilità altre funzionalità**
  - https://learn.microsoft.com/sql/relational-databases/performance/query-store-hints#query-store-hints-and-feature-interoperability

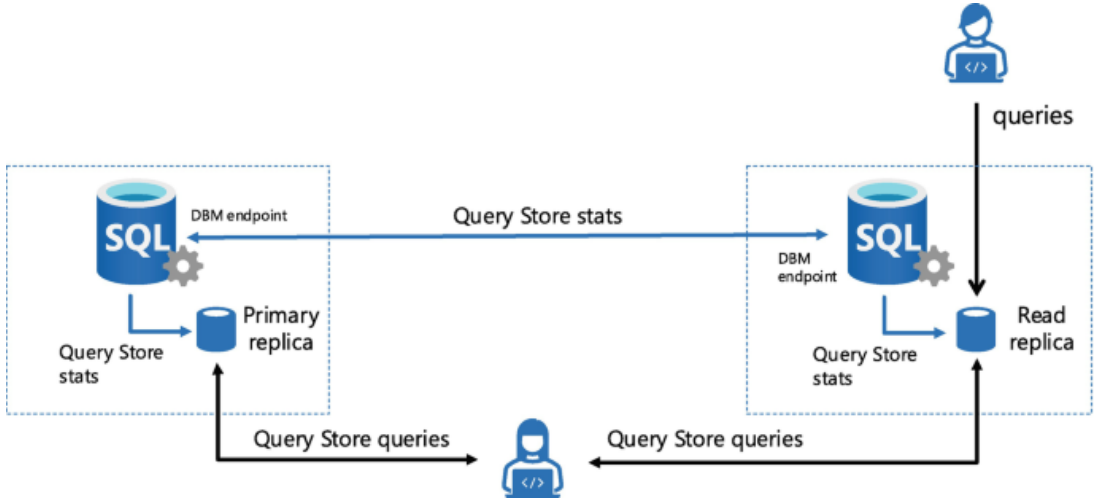
With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
packt

Demo

- Query Store «Hints»

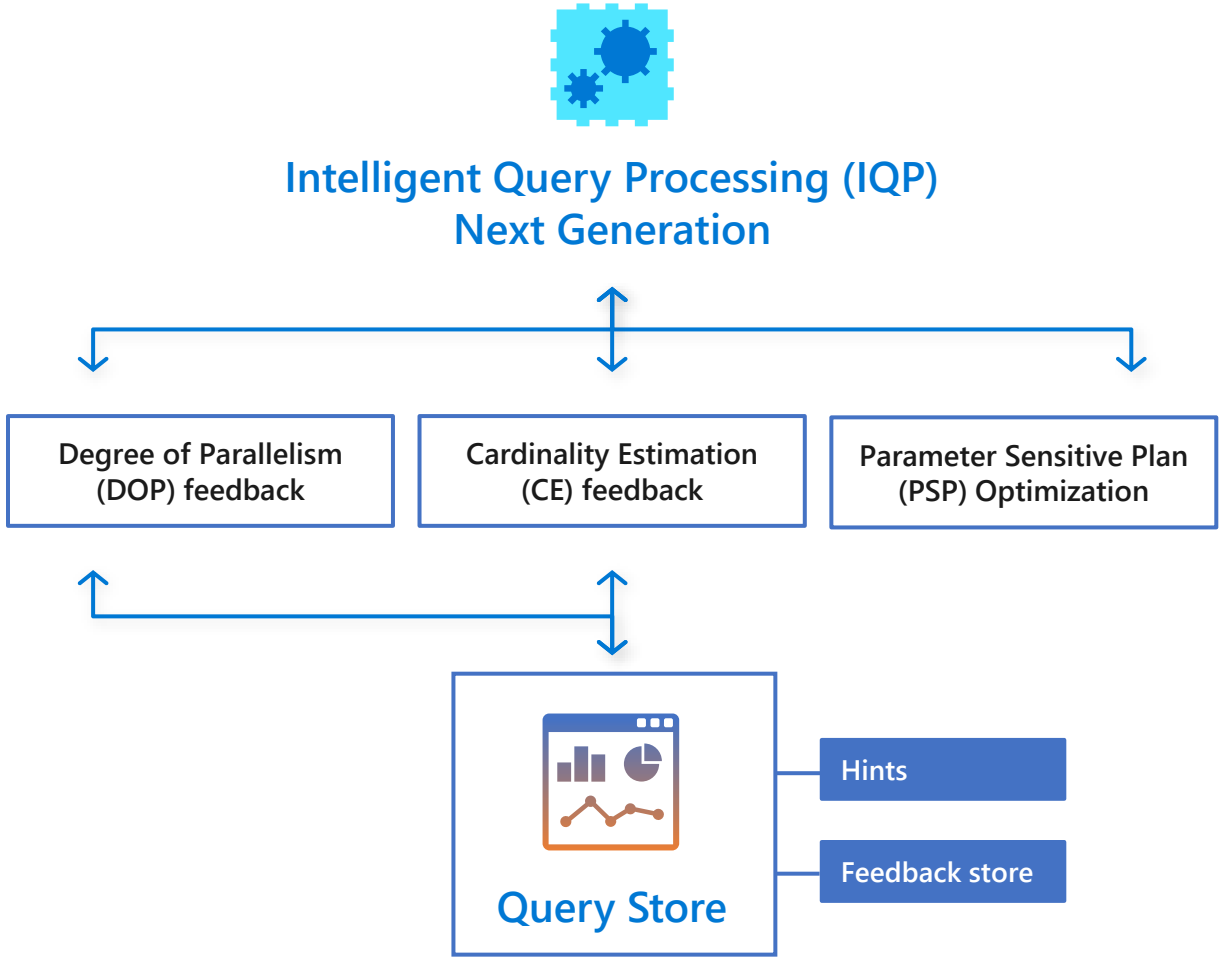
«Query Store» e repliche secondarie AG

- Replica primaria mantiene dati
  - Per tutte le repliche
  - Colonna **`replica_group_id`** solo in
    - **`sys.query_store_replicas`**
    - **`sys.query_store_runtime_stats`**
    - **`sys.query_store_wait_stats`**
- Abilitazione tramite TF 12606
- Maggiori informazioni
  - https://learn.microsoft.com/sql/relational-databases/performance/query-store-for-secondary-replicas



With the support of:

Query Store and IQP



With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
<packt>

DATA SATURDAYS

Intelligent Query Processing

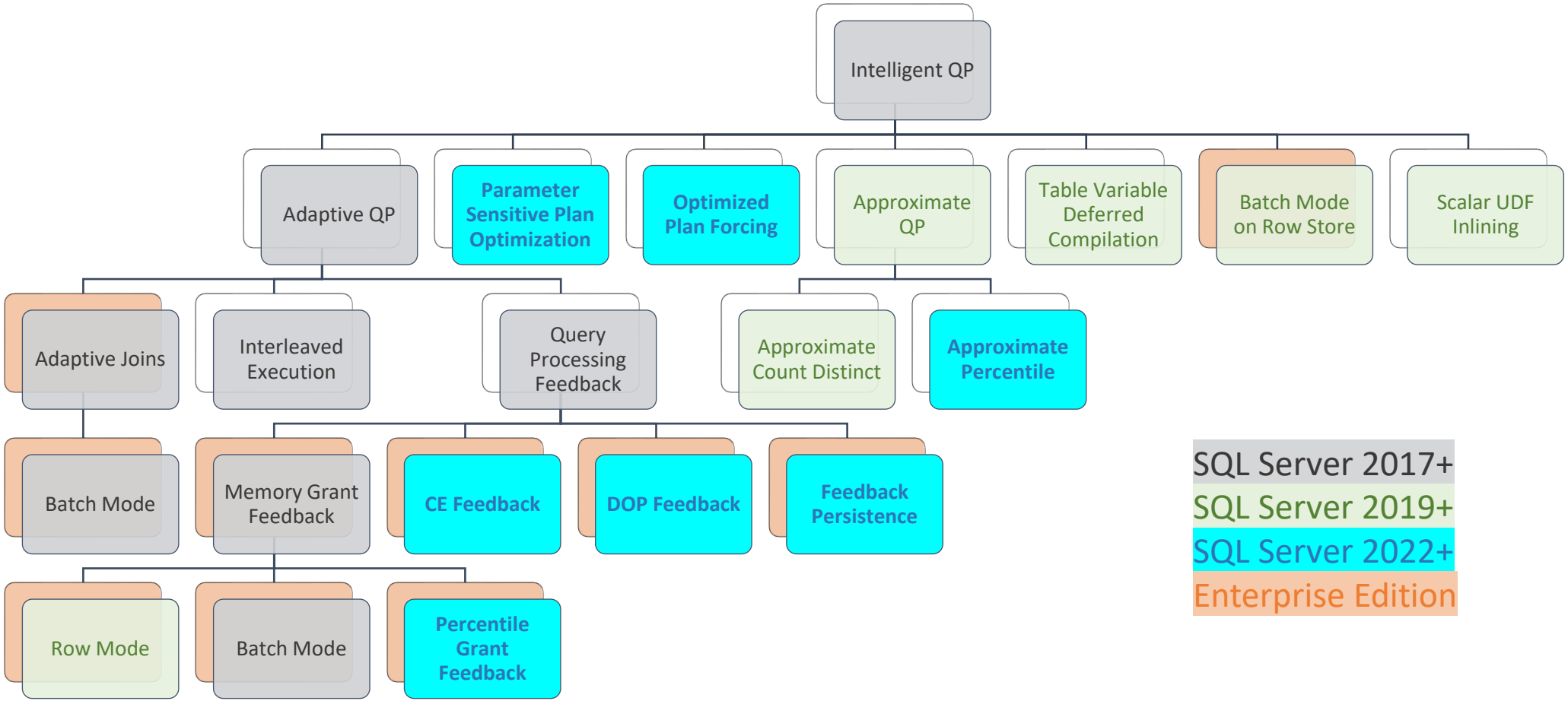
SQL Server 2022 Intelligent Query Processing

With the support of:

innOva community · altitudo · beanTech IT moves your business · fluentis the ERP ready to go live! · QUANTUMDATIS · [stesi] Powered by Innovation · CREDEM · CREDEMTEL · packt

DATA SATURDAYS

Intelligent Query Processing Gen 3



SQL Server 2017+
SQL Server 2019+
SQL Server 2022+
Enterprise Edition

With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
packt

«Execution Modes»

- «Row Mode»
  - Iteratori albero piano esecuzione consumano 1 riga alla volta
  - Modalità esecuzione tradizionale con «Rowstore»
- «Batch Mode»
  - Iteratori albero piano esecuzione consumano batch di righe alla volta
  - Ottimale per operazioni di «scan» (es. aggregati o join tra tabelle grosse)
  - SQL Server 2012: introdotto per sfruttare al meglio indici Columnstore
  - SQL Server 2016/2017: estesi scenari di utilizzo con indici Columnstore
  - SQL Server 2019: estesi scenari di utilizzo con «Rowstore»

«Batch Mode» con «Rowstore»

- Aiuta a ridurre consumo di CPU
- Indice colonnare rimane scelta migliore
  - Per carichi di lavoro OLAP con molto I/O
  - Quando non si può creare (es. impatto su OLTP, funzionalità non supportata)
- Limiti
  - Tabelle «In-Memory» non supportate (solo «Heap» e «B-Tree» su disco)
  - Non usato durante lettura/filtraggio colonne LOB
    - Inclusi «sparse columns sets» e XML

«Batch Mode» con «Rowstore»

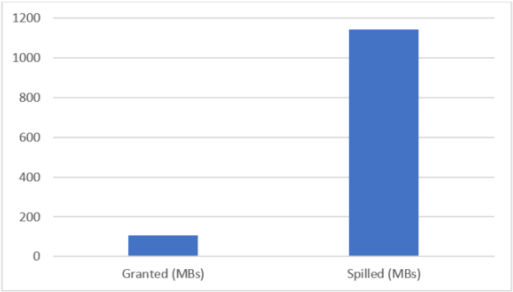
- SQL Server < 2019
  - Alcuni scenari possibili tramite «trucchi» (articoli part1, part2, part3)
- SQL Server 2019+
  - Solo scenari supportati direttamente da «Query Processor»
  - Attivo di default con livello compatibilità database **150+**
  - `ALTER DATABASE SCOPED CONFIGURATION SET BATCH_MODE_ON_ROWSTORE = ON|OFF`
  - `OPTION (USE HINT ('ALLOW_BATCH_MODE'));`
  - `OPTION (USE HINT ('DISALLOW_BATCH_MODE'));`

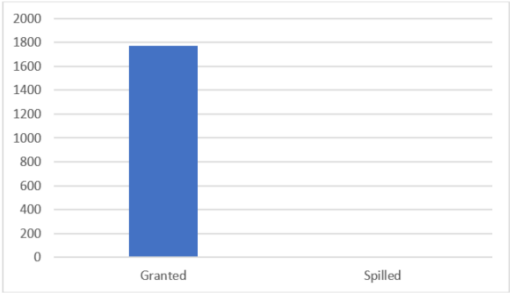
Demo

- «Batch Mode» con «Rowstore»

Memory Grant Feedback

- Problema
  - Troppa poca memoria -> «Spill»
  - Troppa memoria -> inefficienza, possibile attese
- MGF: valutazione post-esecuzione
  - Aggiorna memoria «granted» per i piani in cache
  - Es. più memoria se «spill», meno se «grant» eccesivo
  - Non funziona con OPTION(RECOMPILE)
    - previene «caching» e «memory grant feedback»
- Versioni supportate
  - SQL Server 2017+ «Batch Mode»
  - SQL Server 2019+ «Row Mode»
  - SQL Server 2022+ «Percentile» e persistenza

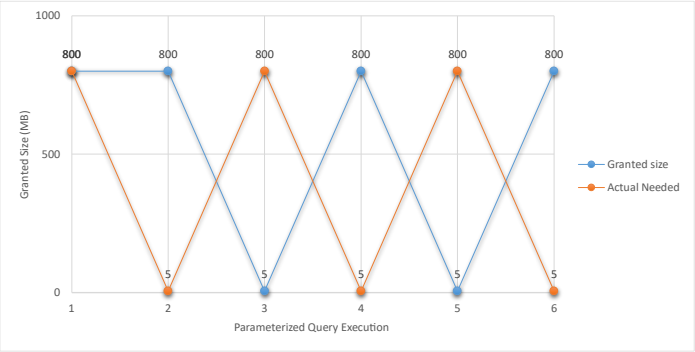


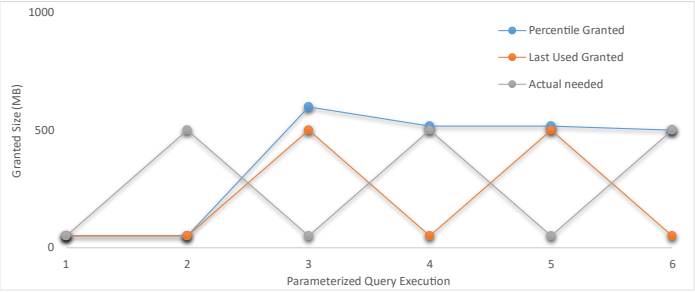


With the support of:

Memory Grant Feedback «Percentile» e Persistenza

- MGF basato su percentile
  - < 2022 disabilitava se valore instabile/volatile
  - Percentile sfavorisce «Spill»
- Persistenza
  - < 2022 non persistente
    - Rimozione da cache (svecchiamento o riavvio)
  - 2022 salvato nel Query Store
    - Abilita MGF anche con OPTION(RECOMPILE)





With the support of:

Controllo «Memory Grant Feedback»

- «Batch Mode»
  - Attivo di default con livello compatibilità database **140+**
  - `ALTER DATABASE SCOPED CONFIGURATION SET BATCH_MODE_MEMORY_GRANT_FEEDBACK = ON|OFF`
  - `OPTION (USE HINT('DISABLE_BATCH_MODE_MEMORY_GRANT_FEEDBACK'));`
- «Row Mode»
  - Attivo di default con livello compatibilità database **150+**
  - ALTER DATABASE SCOPED CONFIGURATION SET **ROW_MODE_MEMORY_GRANT_FEEDBACK** = ON|OFF
  - OPTION (USE HINT ('**DISABLE_ROW_MODE_MEMORY_GRANT_FEEDBACK**'));
- «Percentile»
  - Attivo di default con livello compatibilità database **140+**
  - `ALTER DATABASE SCOPED CONFIGURATION SET MEMORY_GRANT_FEEDBACK_PERCENTILE = ON|OFF`

Troubleshooting «Memory Grant Feedback»

- Scenari query sensibili a parametri
  - Alcune query richiedono «grant» diversi in base a parametri
  - «Memory Grant Feedback» si disabilita se rileva instabilità
- Extended Events per controllare comportamento
  - SQL Server 2017+ **`memory_grant_feedback_loop_disabled`**
  - SQL Server 2019+ **`memory_grant_updated_by_feedback`**
- SQL Server 2019+ proprietà del piano esecuzione
  - **IsMemoryGrantFeedbackAdjusted**
    - No: First Execution, Accurate Grant, Feedback disabled
    - Yes: Adjusting, Stable
  - **LastRequestedMemory**

Demo

- «Memory Grant Feedback»

CE Feedback: problemi con assunzioni

- Correlazione filtri
  - Completamente indipendenti (CE70)
  - Parzialmente correlate (CE120+)
  - Completamente correlati (CE Feedback)
- «Join Containment»
  - Semplice (completamente correlati, CE70)
  - Base (completamente indipendenti, CE120+)
- «Optimizer Row Goal»
  - TOP, IN, EXISTS, FAST, SET ROWCOUNT possono ridurre righe
  - Più righe per distribuzione non uniforme

Limitazioni CE Feedback

- Non viene utilizzato se
  - Livello compatibilità database < 160
  - Sono utilizzati Query Store «Hints»
  - Il piano è forzato tramite Query Store
- Controllo
  - ALTER DATABASE SCOPED CONFIGURATION SET **CE_FEEDBACK** = ON|OFF
  - OPTION (USE HINT ('**DISABLE_CE_FEEDBACK**'));

Demo

- «Cardinality Estimator Feedback»

DOP Feedback: problemi con parallelismo

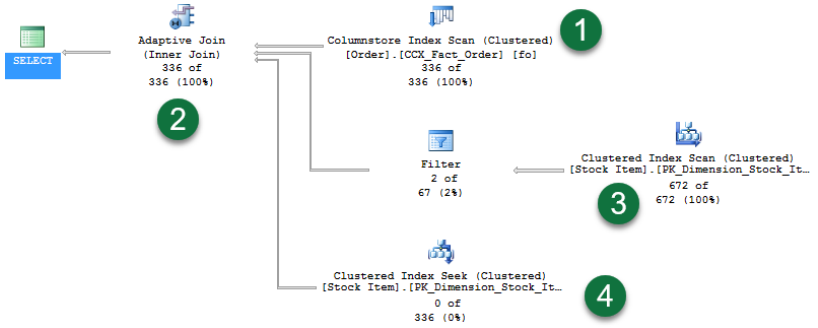
- «Degree of Parallelism»
  - Parallelismo intra-query
  - Massimo configurato per istanza, database (replica), query (hint)
- Grado troppo alto può portare rallentamenti
  - Tempo di coordinamento e/o saturazione CPU
- Analisi basata su tempo di esecuzione e tempi di attesa
  - Esecuzioni ripetute, abbassa grado parallelismo se attese
  - Attese «Buffer Latch», «Buffer IO», «Network IO» ignorate
  - Riduzione attese e aumento concorrenza, anche a discapito durata

Demo

- «Degree of Parallelism Feedback»

«Batch Mode Adaptive Joins»

- «Nested Loop Join» efficiente
  - tabelle input molto piccole
  - una tabella input piccola + indice per «seek» su tabella grande
- «Hash Join» efficiente
  - tabelle di input molto voluminose
  - potenzialmente molta memoria
- «Adaptive Joins»
  - rinvia la scelta dopo scansione primo input



Demo

- «Batch Mode Adaptive Joins»

«Interleaved Execution»

- Problema con funzioni che ritornano tabelle
  - «multi-statement table valued functions» (MSTVF)
  - SQL Server <= 2012 QO assume cardinalità = 1
  - SQL Server 2014/2016 QO assuma cardinalità = 100
- SQL Server >= 2017
  - Inizia ottimizzazione
  - Esegue MSTVF se candidata per «Interleaved Execution»
  - Riprende ottimizzazione con la cardinalità corretta

Demo

- «Interleaved Execution»

Variabili Tabella e Tabelle Temporanee

Area	Tabelle temporanee	Variabili Tabella
Creazione e aggiornamento manuale statistiche	Si	No
Indici	Si	Solo definizioni «in-line»
Vincoli	Si	Solo PRIMARY KEY, UNIQUE e CHECK
Creazione statistiche automatica	Si	No
Creazione e uso di oggetti temporanei in un singolo batch	Compilazione di un comando che referenzia una tabella temporanea che non esiste è differito fino al momento delle prima esecuzione del comando	Un comando che referenzia una variabile tabella è compilato insieme a tutti gli altri comandi prima che uno di questi la popoli, il QO assume cardinalità uguale a 1

With the support of:

Compilazione differita variabili tabella

- Compilazione comando che referenzia variabile tabella
  - SQL Server < 2019 compilato prima che sia popolata con cardinalità fissa = 1
  - SQL Server 2019+ differito a prima esecuzione (come tabelle temporanee)

Demo

- Compilazione differita variabili tabella

Controllo compilazione differita VT

- Attivo di default con livello compatibilità database **150+**
- `ALTER DATABASE SCOPED CONFIGURATION SET` **`DEFERRED_COMPILATION_TV`** `= ON|OFF`
- `OPTION (USE HINT` **`('DISABLE_DEFERRED_COMPILATION_TV')`**`);`

«Scalar UDF In-lining»

- Funzioni utente T-SQL (UDF) che ritornano un singolo valore scalare
- Problemi di prestazioni
  - Invocazione iterativa linguaggio interpretato
    - Per ogni riga, «context switching» molto oneroso durante esecuzione query
    - Comandi eseguiti in isolamento, no ottimizzazioni «cross-statement
  - Esecuzione seriale
    - Parallelismo «Intra-query» inibito
  - Mancanza di costi
    - Storicamente attribuiti solo a operatori relazioni, assunzione costo UDF basso…

«Scalar UDF Automatic In-lining»

- SQL Server 2019+ UDF scalari automaticamente convertite in
  - Espressioni scalari
  - «Subquery» scalari
- Ottimizzazione piano completa (UDF non più visibili)

IQP: esempio «Scalar UDF in-lining»

```
CREATE FUNCTION dbo.discount_price(@price DECIMAL(12,2), @discount DECIMAL(12,2))
RETURNS DECIMAL (12,2) AS BEGIN RETURN @price * (1 - @discount); END
```

```
SELECT L_SHIPDATE, O_SHIPPRIORITY
, SUM(dbo.discount_price(L_EXTENDEDPRICE, L_DISCOUNT))
FROM LINEITEM, ORDERS
WHERE O_ORDERKEY = L_ORDERKEY
GROUP BY L_SHIPDATE, O_SHIPPRIORITY
ORDER BY L_SHIPDATE
```

10GB CCI compressed TPC-H Schema, 2 x CPUs (12 cores), 96GB RAM, SSD storage

	Query senza UDF	Query con UDF (senza in-lining)	Query con UDF (e in-lining)
Tempo esecuzione	1.6 secondi	29 minuti 11 secondi	1.6 secondi

With the support of:

Demo

- «Scalar UDF Automatic In-lining»

Controllo «Scalar UDF Automatic Inlining»

- Attivo di default con livello compatibilità database **150+**
- `ALTER DATABASE SCOPED CONFIGURATION SET` **`TSQL_SCALAR_UDF_INLINING`** `= ON|OFF`
- `OPTION (USE HINT` **`('DISABLE_TSQL_SCALAR_UDF_INLINING')`**`);`
- `CREATE FUNCTION … WITH` **`INLINE`** `= ON|OFF`

Requisiti «Scalar UDF in-lining»

- Può usare uno dei seguenti costrutti
  - DECLARE, SET
    dichiarazione/assegnazione variabili)
  - SELECT
    assegnazione variabili singole/multiple
  - IF/ELSE con livelli annidamento arbitrari
  - RETURN singoli o multipli
  - Chiamate a UDF annidate/ricorsive
  - Operazioni relazionali es. EXISTS, ISNULL
- Nessuna invocazione di funzioni con
  - dipendenza dal tempo es. GETDATE()
  - effetti collaterali es. NEWSEQUENTIALID()
- Utilizzo di EXECUTE AS CALLER (default)
- No riferimenti a
  - variabili tabella
  - parametri «table-valued»
  - tipi dato «user-defined»
- Non compilate nativamente
  - Supporto a «interop»
- No funzione di partizionamento
- Non referenziate da
  - clausole GROUP BY
  - colonne calcolate
  - vincoli CHECK
- No signatures added to it

With the support of:

Troubleshooting «Scalar UDF in-lining»

- Colonna **`is_inlineable`** in **`sys.sql_modules`**
  - Non implica che sarà sempre «in-lined»! (es. 1000 righe di codice)
- Piano di esecuzione
  - Se «in-lined»! , nodo XML **<UserDefinedFunction>** mancante
- Extended Events
  - **`tsql_scalar_udf_not_inlineable`**

Funzioni di aggregazione approssimate

- Precisione vs. risultati rapidi e consumo di memoria ridotto
  - Solo scenari dove i valori precisi non sono necessari!
    - es. dashboards, analisi di trend, «feature selection», ecc.
  - Pensato per grossi volumi di dati e richieste
    - Es. 10 miliardi di righe, 1 utente che consuma 1,5GB di memoria per risultato preciso contro 100 utenti in contemporanea che usano 12MB per un risultato approssimato…
- 2019+ **APPROX_COUNT_DISTINCT**
  - Algoritmo «HyperLogLog» tasso errore <= 2% nel 97% dei casi
- 2022+ **APPROX_PERCENTILE_CONT**, **APPROX_PERCENTILE_DISC**
  - Algoritmo «KLL sketch» tasso errore <= 1,33% nel 99% dei casi

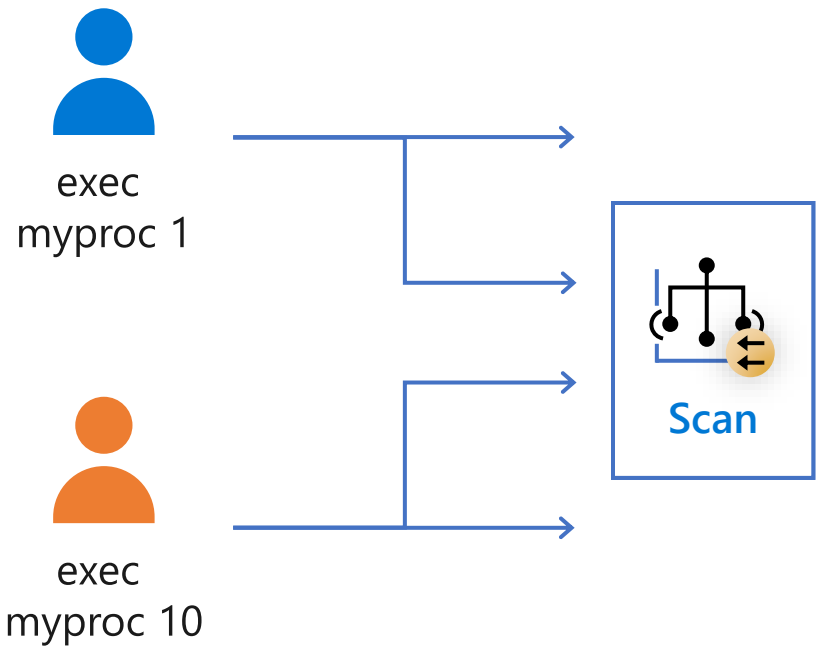
Demo

- APPROX_COUNT_DISTINCT
- APPROX_PERCENTILE_CONT, APPROX_PERCENTILE_DISC

«Parameter Sensitive Plan Optimization»

Before

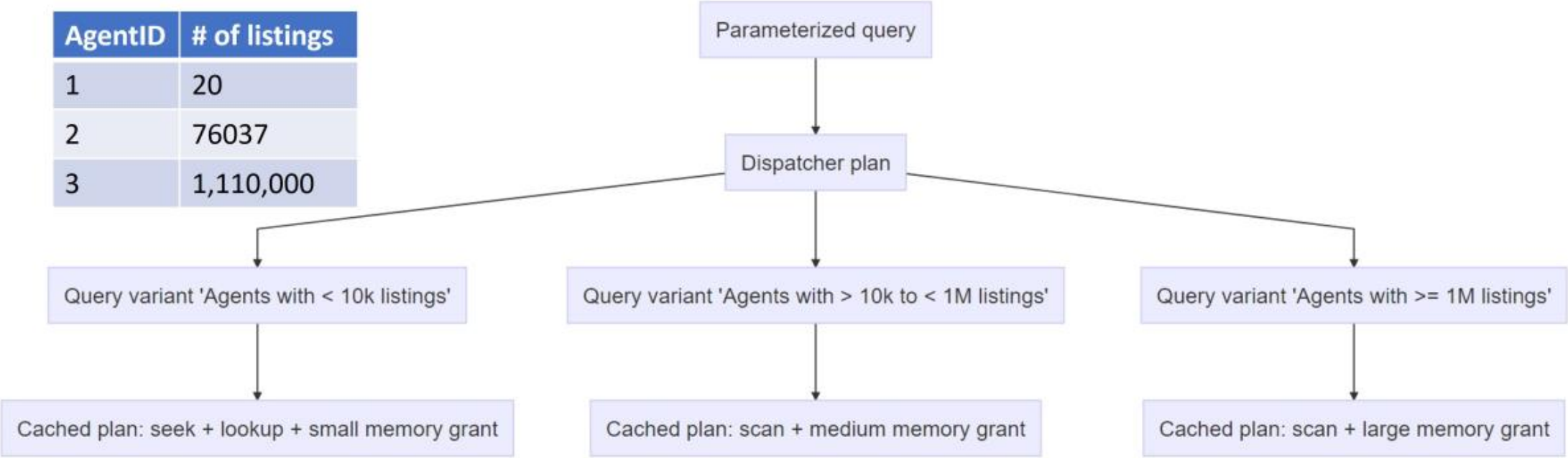
With PSP optimization



With the support of:
innOva community
altitudo
beanTech IT moves your business
fluentis the ERP ready to go live!
QUANTUMDATIS
[stesi] Powered by Innovation
CREDEM
CREDEMTEL
packt

PSP come funziona

AgentID	# of listings
1	20
2	76037
3	1,110,000



With the support of:

Limiti PSP

- Solo operatore di uguaglianza supportato
- Numero finito di varianti
- Numero finito di parametri
- Possibile piano non ottimale per tutti valori di una variante

Demo

- «Parameter Sensitive Plan Optimization»

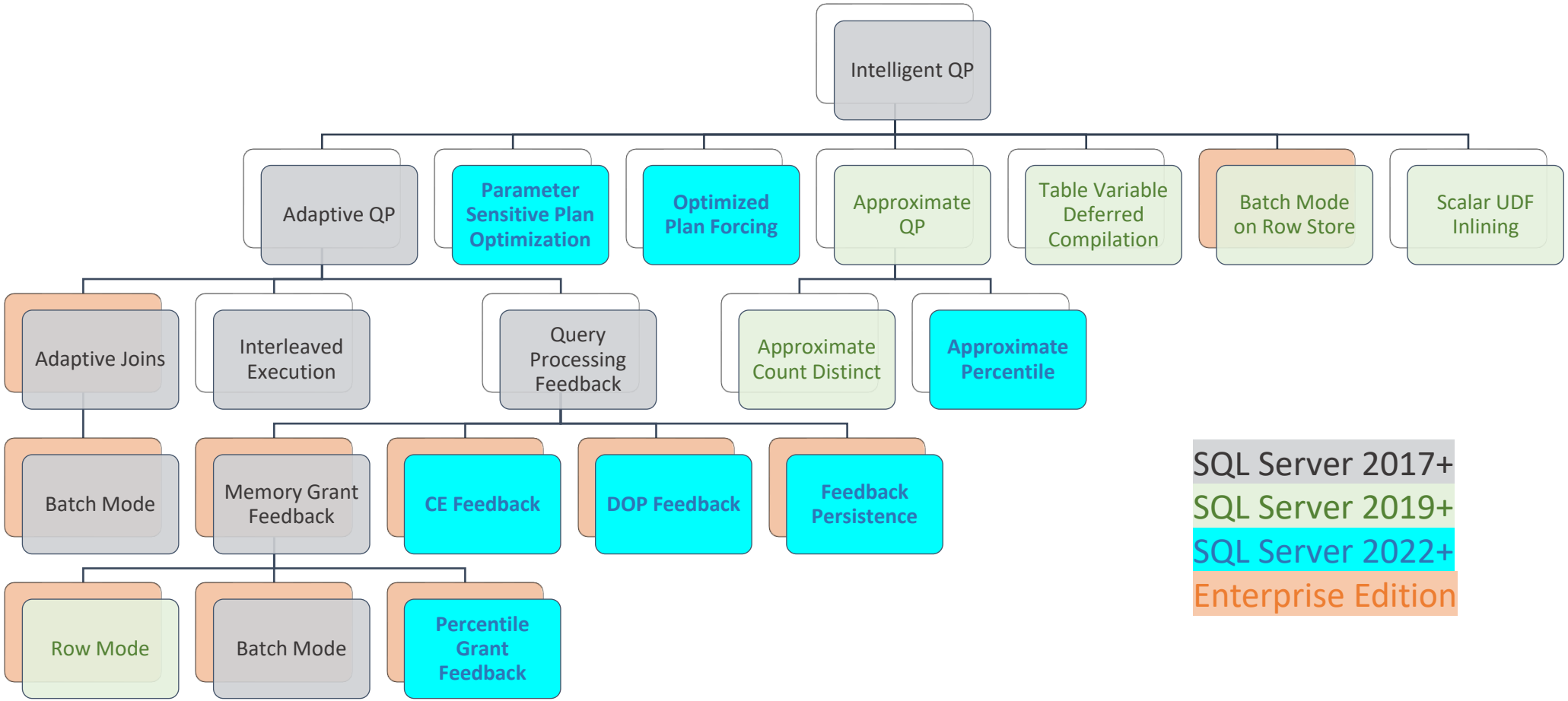
«Optimized Plan Forcing»

- Query con tempi di compilazione alti sono candidate
- «Replay Script» incluso nel piano di esecuzione
  - Salvato nel Query Store come piano forzato
  - Fa risparmiare tempo di compilazione Query
    - Forzatura non significa che non debba essere compilata
  - Utile per evitare «compile storm»

Demo

- «Optimized Plan Forcing»

Review Intelligent Query Processing Gen 3



SQL Server 2017+
SQL Server 2019+
SQL Server 2022+
Enterprise Edition

With the support of:

Modifiche non protette da livello compatibilità

SQL Server 2022

- Approximate Percentile
  - PERCENTILE_CONT
  - PERCENTILE_DISC
- Optimized Plan Forcing
  - Persists compile steps
  - Reduce compilation overhead

dbcompat 140+

- Memory Grant Percentiles
  - Smooth oscillation
- Memory Grant Feedback Persistence
  - Survive cache eviction and restart

dbcompat 160

- Parameter Sensitive Plan (PSP) Optimization
- Cardinality Estimation (CE) Feedback
- Degree of Parallelism (DOP) feedback

With the support of:

Risorse

- https://aka.ms/querystore
- https://aka.ms/iqp
- https://aka.ms/cefeedback
- https://aka.ms/dopfeedback
- https://aka.ms/pspopt

Risorse SQL Server 2022

Learn more about SQL Server 2022
aka.ms/sqlserver2022

Download SQL Server 2022
aka.ms/getsqlserver2022

Get our decks
aka.ms/sqlserver2022decks

What's new for SQL Server 2022
aka.ms/sqlserver2022docs

Read the book
aka.ms/sql2022book

Try out the workshop
aka.ms/sql2022workshop

Try our demos
aka.ms/sqlserver2022demos

Watch Microsoft Mechanics
aka.ms/sqlserver2022mechanics

Don't miss us on Data Exposed
aka.ms/dataexposed

Read the SQL Server 2022 Blog Series
aka.ms/sqlserver2022blogs

Take the Microsoft Learning Path
aka.ms/learnsqlserver2022

With the support of: